

Local AI Orchestrator

System Guide and Architecture

Private, local-first executive work orchestration on macOS
v1 scaffold — clarification implemented, execution graph in progress

Generated: 2026-05-17

1. What this system is

A private, local-first AI work orchestrator running entirely on your Mac. It is **not a chatbot**. It is a system that takes vague work requests ("prepare me for tomorrow," "summarize the board notes"), turns them into structured *work packets*, asks you strong clarifying questions until the request is concrete and safe to execute, scores readiness on nine dimensions, and — eventually — executes the well-scoped work overnight using local Ollama models.

The design intent is two-fold:

- **Clarify before doing anything heavy.** Vague input never produces low-quality output; it produces questions.
- **Keep everything on your machine.** No cloud calls unless every one of six policy gates passes. No external sends in v1. No service exposure beyond localhost.

What it is NOT

- Not a chatbot — for conversations, use Open WebUI at <http://localhost:3000>.
- Not multi-user — single Mac, single operator.
- Not for sending anything externally — email, Slack, calendar invites, GitHub PRs are blocked at the safety layer in v1.
- Not auto-cloud — Claude requires six gates; the day-unlock switch does not open them.
- Not running models in the background all day — DAY_MODE blocks every local-* model group unless you deliberately unlock.

2. Mental model

The system is built around one loop, repeated as often as you have work:

Stage	What happens	Who acts
1. Capture	You give it a title and a vague description.	You
2. Clarify	The system builds an initial work packet, scores readiness on 9 dimensions, and generates up to 7 strong questions.	System
3. Answer	You respond in a markdown file or via the panel.	You
4. Re-score	The system parses your answers, rescores, and either marks the packet READY_FOR_OVERNIGHT (≥ 0.85) or REQUIRES_CLARIFICATION (< 0.65).	System
5. Execute	Overnight, in NIGHT_MODE, the execution graph reads sources, calls local systems (if configured), passes through a safety gate, evaluates, and produces a final output.	System
6. Review	You read the output in <code>~/LocalAI/output/<date>/</code> and the Obsidian vault; approve memory candidates for the long-term memory.	You

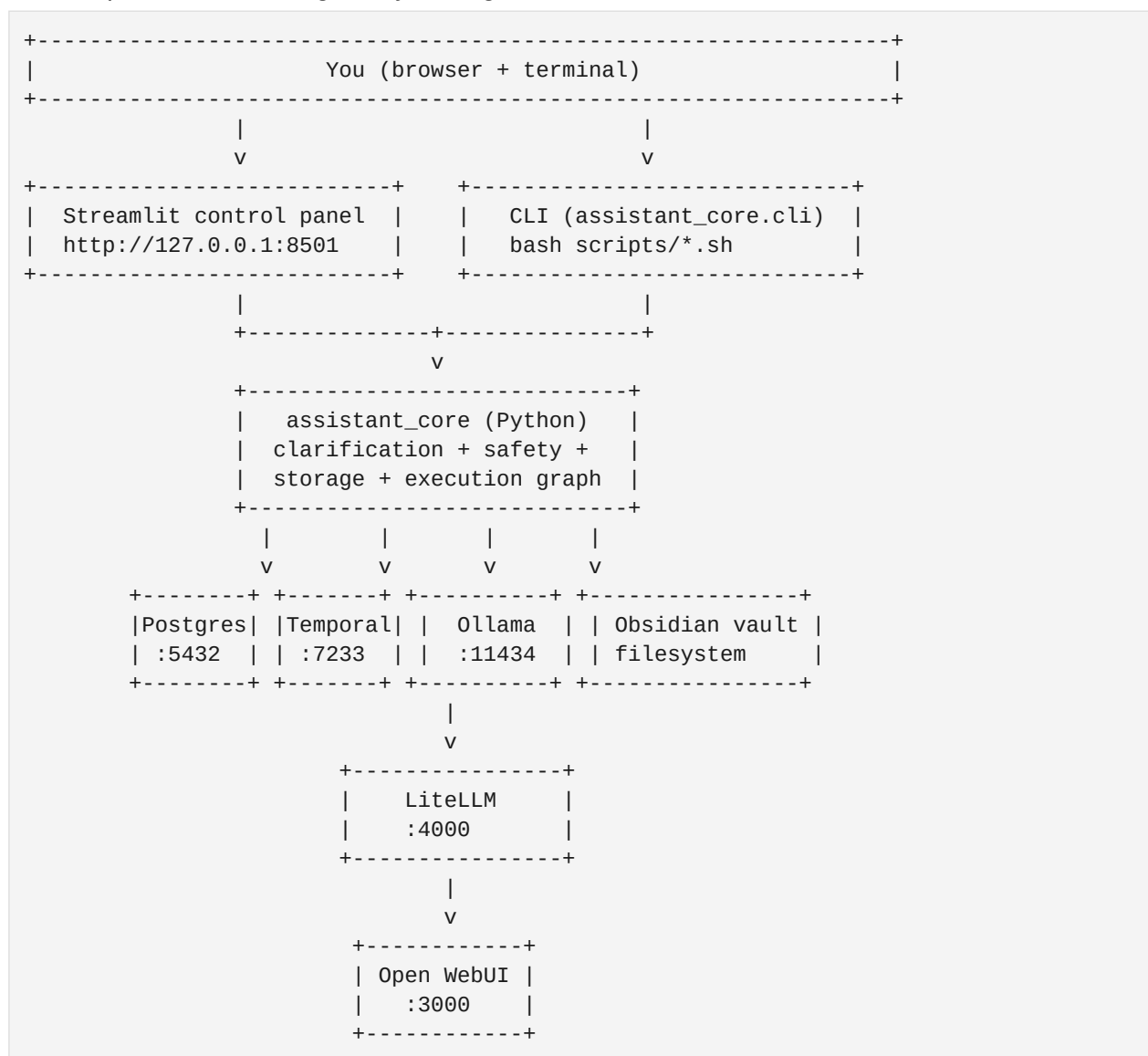
Readiness scoring (nine dimensions)

Dimension	Weight	What it captures
objective	0.18	Is the desired outcome explicit?
output_format	0.14	Is the output shape specified?
sources	0.14	Which folders/files are in scope?
privacy_cloud_policy	0.14	Is cloud explicitly allowed or forbidden?
quality_threshold	0.12	What quality bar to optimize for?
audience	0.10	Who is the output for?
assumption_policy	0.08	May the system infer when uncertain?
escalation_policy	0.06	When should it stop and ask?
success_criteria	0.04	What makes the work done?

Thresholds: ≥ 0.90 = READY_HIGH_CONFIDENCE; ≥ 0.85 = READY_FOR_OVERNIGHT; $0.65\text{--}0.84$ = NEEDS_CLARIFICATION; < 0.65 = UNDERDEFINED. High-stakes packets (board, CEO, legal, investor) auto-detect and require ≥ 0.90 .

3. Architecture at a glance

Nine components, all running locally, talking to each other on 127.0.0.1.



Solid lines: required. LiteLLM and Open WebUI are optional in v1 — the panel can talk to Ollama directly through `assistant_core.llm.ollama_admin`.

4. Components: what each does and why

Python orchestrator (assistant_core)

The brain. Pure Python, no framework.

What it does

Holds the deterministic clarification logic, the safety gates, the work-packet schema, the storage layer, and the (scaffolded) execution graph. Has no opinion about UI — both the Streamlit panel and the CLI call into it. Tested in isolation via pytest.

Why this tool

Pure Python keeps the core auditable, easy to reason about, easy to test. Pydantic for schemas (validation at type boundaries). YAML for human-editable config. Psycopg for direct Postgres I/O without an ORM layer that hides intent.

Where to look

```
assistant_core/{clarification,llm,memory,execution,temporal_app}/
```

Streamlit control panel

Local web UI on :8501.

What it does

Renders the status strip (mode, day unlock, services up/down), seven tabs (Dashboard, Create Work Packet, Clarification, Models, Schedule, Execution, Artifacts), and is the only UI surface for safe model load/unload.

Why this tool

Streamlit lets us iterate in pure Python without writing a frontend. Tabs are stateless re-renders — safe by construction. Editing config or invoking Ollama always goes through assistant_core, never bypasses the gate.

Where to look

```
app/control_panel.py
```

PostgreSQL

Operational state store.

What it does

Holds work_packets, clarification_questions, execution_runs, model_calls, evaluations, artifacts, memory_candidates. Migrated automatically on first container start (db/migrations/001_init.sql is mounted into Postgres' init dir).

Why this tool

Postgres because we want durable, queryable state with relational integrity across the seven tables — work packets relate to questions, runs, model calls, and evaluations. JSON columns for cloud/source/output policy let us evolve without migrations.

Where to look

```
db/migrations/001_init.sql + assistant_core/storage.py
```

Temporal

Durable workflow engine on :7233 (UI on :8233).

What it does

Manages overnight workflows: clarification, execution, manual resume. Survives process crashes; can pause and resume long-running work. The worker process (scripts/start_temporal_worker.sh) executes activities defined in assistant_core/temporal_app/.

Why this tool

Overnight work runs for hours, may hit transient model failures, and needs to be resumable. Temporal handles retries, timeouts, idempotency, and audit trail out of the box. Cron + bash would be fragile at this scale.

Where to look

```
assistant_core/temporal_app/{workflows,activities,worker,client}.py
```

Ollama

Local model runtime on :11434.

What it does

Pulls models to ~/.ollama/models on disk, loads them into Metal VRAM on demand, serves inference via HTTP. The orchestrator never bypasses Ollama for local model calls — it always goes through this daemon.

Why this tool

Ollama is the de facto local-model runtime on macOS Metal. Native Apple Silicon support, good keep-alive semantics, simple HTTP API, library of pre-quantized models. Lighter and easier to operate than running llama.cpp directly.

Where to look

```
config/models.yaml maps groups -> Ollama tags
```

LiteLLM

Model router and gateway on :4000.

What it does

Translates a model-group name (local-main, local-coder, etc.) into the right Ollama tag or cloud endpoint. Configured with local-only fallbacks; cloud is intentionally not chained behind local.

Why this tool

LiteLLM gives us a single OpenAI-style API for code that wants to swap models later, with fallback rules expressed declaratively. We keep the cloud route in the config but disable cloud at the safety layer so no blind escalation can happen.

Where to look

```
config/litellm.yaml.example + assistant_core/llm/litellm_client.py
```

Open WebUI

ChatGPT-style chat against local models, on :3000.

What it does

A separate tool that lives next to the orchestrator. Talks directly to Ollama. Provides conversational access when you want it; the orchestrator deliberately does not offer chat.

Why this tool

Sometimes you want to chat with a model. Open WebUI is a polished, mature tool for that. Co-installing it via docker-compose lets you have both surfaces.

Where to look

```
docker-compose.yml — service: open-webui
```

Obsidian vault

Human-readable long-term notes.

What it does

The system writes outputs and memory candidates into `~/Obsidian/LocalAI-ChiefOfStaff/`. Daily briefs, work packets, decisions, meeting notes, memory review queue. You read and curate these directly in Obsidian.

Why this tool

Obsidian's plaintext markdown vault is durable, indexable, and yours. No lock-in. The orchestrator writes; you review. The vault becomes your long-term knowledge layer.

Where to look

```
~/Obsidian/LocalAI-ChiefOfStaff/{01_Daily_Briefs, 02_Work_Packets, ...}
```

Chroma (vector store)

Local semantic retrieval target.

What it does

Scaffolded but not yet wired. Will hold embeddings of selected outputs and vault content for retrieval-augmented work.

Why this tool

Chroma is simple, file-backed, no separate daemon. Right tradeoff for a single-user local system. Persistent client at `~/LocalAI/state/chroma`.

Where to look

```
assistant_core/memory/vector_store.py
```

Docker Compose

Service orchestration for Postgres / Temporal / Open WebUI.

What it does

One file, four services. Containers expose ports only on 127.0.0.1. Named volumes persist data across container recreates. The migration SQL is mounted into Postgres' init directory so the schema is created on first run.

Why this tool

Docker keeps these three services consistent across machines without ceremony. They're stateful, multi-process, and have non-trivial init — containers handle that cleanly.

Where to look

```
docker-compose.yml
```


5. The safety system in depth

The orchestrator is built around the assumption that you do not want the system to make decisions you wouldn't make yourself. Every model call, every cloud fallback, every external write passes through a deterministic gate in `assistant_core/safety.py`. Gates are written as *fail-closed* — an unknown tag, an empty env var, or a missing flag all default to **blocked**.

The five gate functions

Function	What it gates
<code>assert_cloud_allowed</code>	Six checks before any Claude call: <code>cloud_fallback_enabled</code> , <code>work_packet_cloud_allowed</code> ,
<code>assert_no_external_write</code>	Always blocks in v1. Used at any future email/Slack/API boundary.
<code>assert_original_file_write_allowed</code>	Refuses writes outside the configured roots (output, archive, logs, work_packets, vault).
<code>assert_heavy_execution_allowed</code>	Refuses model execution in DAY_MODE unless day unlock is active or <code>manual_override=</code>
<code>assert_model_allowed</code>	Top-level entry point. Rejects cloud-claude-opus outright; then delegates local groups to a

DAY_MODE / NIGHT_MODE / MANUAL_RESUME

Mode is a single env var, `LOCALAI_MODE`. The default is `DAY_MODE`. The overnight runner refuses to start unless `NIGHT_MODE` is set; the launchd plist sets it automatically when it fires. The third value, `MANUAL_RESUME`, is reserved for user-driven re-runs of previously paused workflows.

The day-unlock switch

`DAY_MODE` blocks all four local-* model groups (local-main, local-secondary, local-coder, local-reasoner). To deliberately work with models during the day, the day-unlock switch flips a single boolean. Two channels:

- **Sentinel file** — `~/LocalAI/state/day_unlock.flag`. Created by `scripts/day_unlock.sh` with a timestamp, hostname, and reason. Persists across terminal closes and reboots.
- **Environment variable** — `LOCALAI_DAY_UNLOCK=true`. Per-shell, ephemeral.

Either being on opens the gate. Cloud and external-write gates are **not** affected. The Streamlit panel renders a red banner with the flag contents while unlock is `ACTIVE` so the state is impossible to miss.

Full design rationale: `docs/day_unlock.md`. Locking again: `bash scripts/day_lock.sh`.

6. Auto-execution (launchd nightly job)

Default state: **OFF**. The orchestrator can be scheduled to run overnight via macOS launchd, but the plist is never armed automatically. Arming requires three deliberate steps in the terminal.

The four states

Project plist	Installed in LaunchAgents	Loaded in launchctl	State
no	no	no	fully off, nothing installed
yes	no	no	off (default after install_core)
yes	yes	no	off, installed but not loaded
yes	yes	yes	ARMED, will run nightly at night_mode_start

Arming (three deliberate steps)

```
# 1. Verify the configured time is what you want.
#   Edit it in the panel's Schedule tab or in config/assistant.yaml.

# 2. Regenerate the plist with that time:
bash scripts/create_launchd_plist.sh

# 3. Install into LaunchAgents and load:
LOCALAI_WRITE_LAUNCHD=true bash scripts/create_launchd_plist.sh
launchctl load ~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist
```

Disarming

```
launchctl unload ~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist
# Optionally:
rm ~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist
```

Full design rationale: docs/auto_execution.md.

7. End-to-end data flow

From a vague request to overnight output, in nine steps:

Step	What happens	Storage
1	You: bash scripts/create_work_packet.sh "Title" "Description"	—
2	build_initial_work_packet builds a WorkPacket pydantic model.	in memory
3	score_readiness assigns 0/1 per dimension, computes weighted score.	in memory
4	generate_questions emits up to 7 ranked clarification questions.	in memory
5	create_work_packet + save_questions write the row and questions to postgres.	Postgres work_packets, clarification_questions
6	You answer in a markdown file. bash scripts/answer_questions.sh &PAID;S.	&PAID;S.
7	update_packet_from_answers extracts fields. Rescore. If READY, packet updates.	Postgres update work_packets
8	Overnight runner (manual or via launchd at NIGHT_MODE) finds READY packets, submits Temporal workflows.	Postgres packets, submits
9	Workflow body (scaffolded) reads sources, calls models, evaluates, writes output to storage, crafts output details, and publishes.	Filesystem output, Postgres Local, output details, and Obs

Step 9 is still scaffolded as of this writing. The workflow is wired and the Temporal worker accepts the submission, but the activity body returns a placeholder dict instead of running the 18-step pipeline. That is the next feature-work chunk.

8. What's implemented vs scaffolded

Capability	State	Notes
Deterministic clarification flow	Implemented	Work packet builder, 9-dim readiness, question generator. 1
Postgres storage layer	Implemented	7 tables, schema migration applied, CRUD for packets/quest
Safety gates (5 functions)	Implemented	Tests parameterized over every local-* group and DAY/NIGH
Day-unlock switch	Implemented	File + env channel, visible banner, fail-closed. docs/day_unlo
Streamlit control panel	Implemented	Header strip, 7 tabs, Models tab, Schedule tab, config editor
Models tab (load/unload/test)	Implemented	Gate-checked. CLI workflow: Load -> Quick test -> Unload.
Schedule tab (time editing)	Implemented	Atomic config writes, validation, launchd-status mirror.
Ollama integration	Implemented	Pull, list, load, unload, generate. ollama_admin.py.
Temporal worker + workflow definitions	Implemented (skeleton)	Workflows defined, activities defined, but bodies return place
LiteLLM routing	Implemented (config only)	Started via scripts/start_litellm.sh. Used by future model-call
Launchd plist generation	Implemented	Reads night_mode_start from config; arming stays a manual
Overnight execution graph (18 steps)	Scaffolded	EXECUTION_STEPS list exists. Activity body returns a stub
Quality-gated local retry loop	Scaffolded	evaluators.py exists with deterministic fallback. Retry orches
Pause/resume UI	Scaffolded	Buttons disabled in panel. DB statuses (PAUSED, RESUME
Semantic memory indexing	Scaffolded	Chroma client exists. No indexing workflow yet.
Claude cloud-review path	Scaffolded	Gates implemented; no caller wired.
Drafted messages	Scaffolded	Output template exists; nothing fills it yet.

9. File system layout

Project repository

```
local-ai-orchestrator/
  AGENTS.md          rules for human/AI editors
  ONBOARDING.md      practical usage guide (separate PDF)
  README.md          setup + ports + safety summary
  app/control_panel.py Streamlit UI
  assistant_core/     Python package
    clarification/    work packet builder, readiness, questions
    execution/        18-step pipeline (scaffolded)
    llm/              ollama_admin, litellm_client, model_policy
    memory/           Obsidian + Chroma writers
    temporal_app/     worker, workflows, activities
    safety.py         5 gate functions
    schemas.py        pydantic models for the whole system
    storage.py        Postgres CRUD
    config.py         yaml config loader
    config_writer.py  atomic config edits
    scheduler_status.py launchd + service health
  config/
    assistant.yaml    day/night times, thresholds, paths
    models.yaml       group -> Ollama tag mapping
    memory.yaml       vector store settings
    litellm.yaml      LiteLLM router config (gitignored copy)
  db/migrations/      SQL applied on Postgres first boot
  docs/               deep-dive design docs
  examples/           sample answers, sample packet
  launchd/            generated plist (gitignored)
  scripts/            bash entrypoints for every operation
  tests/              pytest, isolated from real state
  docker-compose.yml  postgres + temporal + temporal-ui + open-webui
```

Data directories (outside the repo)

```
~/LocalAI/
  inbox/
    notes/           your input notes
    transcripts/      call/meeting transcripts
    docs/            documents
    csv/             tabular sources
    cloud_review/     files marked for cloud review (Claude gate path)
  output/<date>/      generated artifacts (briefs, priorities, etc.)
  archive/           processed inputs
  logs/              runtime logs (incl. launchd stdout/stderr)
  state/             operational state (incl. day_unlock.flag)
  work_packets/      packet-specific files

~/Obsidian/LocalAI-ChiefOfStaff/
  00_Inbox/Memory_Review/ memory candidates awaiting your approval
  01_Daily_Briefs/        morning briefs
  02_Work_Packets/        packet notes
```

03_Projects/	project pages
04_Meetings/	meeting notes
05_Decisions/	logged decisions
06_Stakeholders/	people pages
07_Playbooks/	reusable procedures
08_Prompts/	curated prompts
09_System_Logs_Summaries/	run summaries
99_Archive/	historical

10. Database schema

Seven tables in the localai database, all created from db/migrations/001_init.sql:

Table	Purpose
work_packets	One row per packet: title, objective, status, readiness, policies (jsonb), raw and structured request
clarification_questions	Per-packet questions across rounds; each can be answered or unanswered.
execution_runs	One row per overnight invocation, with Temporal workflow id, mode, start/end timestamps.
model_calls	Audit row per local-model or cloud call: task type, group, actual tag, sizes, success/error.
evaluations	Quality / grounding / completeness / contradiction scores from the evaluator, with recommended
artifacts	File-level pointers to generated outputs and their Obsidian copies.
memory_candidates	Items extracted from outputs as candidates for long-term memory; require user approval.

All tables use UUID primary keys. JSONB columns on work_packets allow policy evolution without migrations. Foreign keys enforce relational integrity. Indexes cover status, work_packet_id joins, and timestamp ranges.

11. Configuration

Three YAML files in config/, plus a small set of env vars.

config/assistant.yaml — main tunables

Key	Default	Meaning
day_mode_start	07:30	When the day window begins (editable in Schedule tab).
night_mode_start	01:30	When night begins; drives the launchd plist.
night_mode_end	07:00	Cutoff for night work.
clarification_required	true	Skip clarification only with explicit override.
minimum_readiness_for_overnight	0.85	Threshold for READY_FOR_OVERNIGHT.
minimum_readiness_for_high_stakes	0.90	Threshold when packet.high_stakes is true.
cloud_fallback_enabled	false	Master switch for cloud. Even with true, six more gates apply.
daily_cloud_budget_usd	5.00	Hard cap on Anthropic spend per day.
max_local_attempts	2	Retry budget through local-main -> local-secondary.
max_chars_per_chunk	12000	Source-file chunking for long inputs.
temperature, top_p	0.2 / 0.9	Conservative sampling defaults for execution.

config/models.yaml — group to tag

```
local-main:      qwen3:30b-a3b
local-secondary: llama3.3:70b
local-coder:     qwen3-coder:30b
local-reasoner:  deepseek-r1:8b
cloud-claude-opus: claude-opus-4-7
```

Environment variables

Variable	Effect
LOCALAI_MODE	DAY_MODE (default), NIGHT_MODE, or MANUAL_RESUME
LOCALAI_DAY_UNLOCK	Truthy value opens the day gate for this shell only
LOCALAI_MANUAL_RESUME	Lets run_ready_overnight.sh proceed outside NIGHT_MODE
LOCALAI_WRITE_LAUNCHD	Truthy = create_launchd_plist.sh installs into LaunchAgents
LOCALAI_POSTGRES_URL	Override DB connection (defaults to local Docker)
ANTHROPIC_API_KEY	Required for cloud-claude-opus calls (plus 5 other gates)

12. Glossary

Term	Definition
Work packet	The orchestrator's unit of work. A structured representation of a request with objective, sources,
Readiness score	Weighted sum of nine binary dimension checks, in [0, 1]. Threshold 0.85 to be eligible for overnight
Clarification round	One pass of: build packet -> score -> generate questions -> wait for answers.
DAY_MODE	Mode in which all local-* model groups are blocked at the safety gate unless day_unlock is active
NIGHT_MODE	Mode in which local model calls are allowed. Required by the overnight runner.
Day unlock	An explicit, visible override that lets local models load in DAY_MODE. Implemented as a flag file
Model group	A logical role (local-main, local-coder, etc.) that maps to a specific Ollama tag in config/models.y
Keep-alive	Ollama's per-model VRAM retention timer. Our Load button uses 30m; Quick test uses 5m; Unlo
Cloud-review path	The Claude Opus route, gated by six policy checks. Files must be in ~/LocalAI/inbox/cloud_revie
Auto-execution	The launchd job that runs run_ready_overnight.sh at night_mode_start. Off by default; arming is
Manual override	A per-call boolean (manual_override=True) that bypasses the day gate. Used in tests and direct

End of System Guide. For the practical usage walkthrough, see the companion Onboarding PDF.